TaskCloud

Cloud architecture and deployment plan

Course project, DS3203A Cloud Computing · Team: Aadesh (admin), Sanchit, Aatyush, Krushna · Region: Asia Pacific (Mumbai), `ap-south-1` · 6 November 2026

TaskCloud is a serverless app that lets students manage tasks, track progress and turn one typed prompt into a saved study plan. There are no servers to run. CloudFormation creates everything, so three team members only run a script and submit their screenshots and logs.

What the design adds to the original idea

Decision	Why
Agent runs in Lambda, using the Bedrock Converse API with tools	Amazon closed the older Bedrock Agents service to new accounts on 30 July 2026. A small agent loop in Lambda is the simplest option that CloudFormation can deploy.
Add API Gateway	The browser needs one HTTPS entry point to reach Lambda, with login checks and rate limits.
Add Cognito, with public sign-up off	Without login, anyone who finds the API could spend the team's Bedrock budget or read another student's tasks.
Deploy from CloudShell	Team members log in to the console and run scripts there, so nobody needs long-lived access keys on a laptop.

Services used

Service	Job in TaskCloud	Deployed by
AWS Amplify Hosting	Hosts the static website over HTTPS	**Krushna**
Amazon Cognito	Logs students in and issues a signed token (JWT)	**Sanchit**
Amazon DynamoDB	Stores tasks and progress for each student	**Sanchit**
API Gateway (HTTP API)	Single HTTPS API, checks the token, limits request rates	**Aatyush**
AWS Lambda (Python 3.13)	`tasks-api` for task operations, `attachment-indexer` for S3 events	**Aatyush**
Amazon S3	Stores files attached to tasks, such as study notes	**Aatyush**
Amazon Bedrock + Lambda	`planner-agent` turns a prompt into tasks using an AI model	**Aadesh**
Systems Manager	Parameter Store keeps the model ID, so the model can change without a code change	**Aadesh**
CloudFormation, IAM, CloudTrail, CloudWatch, Budgets	Infrastructure as Code, access control, audit trail, logs and cost alerts	**Aadesh**

1. Architecture

The browser talks to four AWS services directly. All task data and AI planning sit behind API Gateway, which rejects any request without a valid login token.

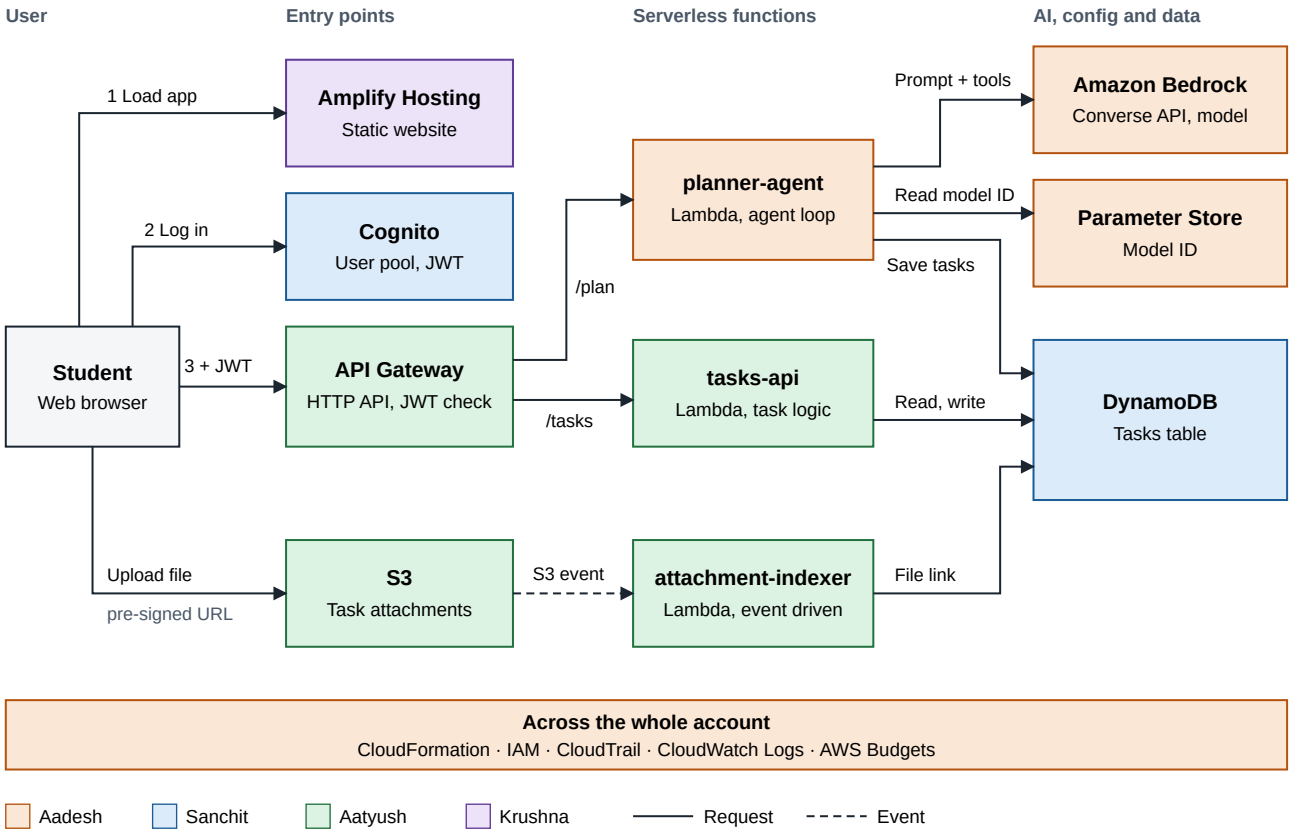


Figure 1. Box colour shows who deploys each part.

The three main flows

Flow	What happens
Manage a task	The browser sends the request with its JWT. API Gateway validates the token, then `tasks-api` uses the verified user ID to read or write DynamoDB.
Plan from a prompt	The student types a prompt. `planner-agent` asks the model to choose tasks, then validates and saves them. Section 2 shows each step.
Attach a file	`tasks-api` returns a pre-signed URL valid for 5 minutes. The browser uploads directly to S3, which triggers `attachment-indexer` to link the file to its task.

2. The AI planning agent

It is an agent because the model chooses which tool to use next, and the Lambda function carries out that choice. The model never touches the database directly. Lambda checks every tool request first.

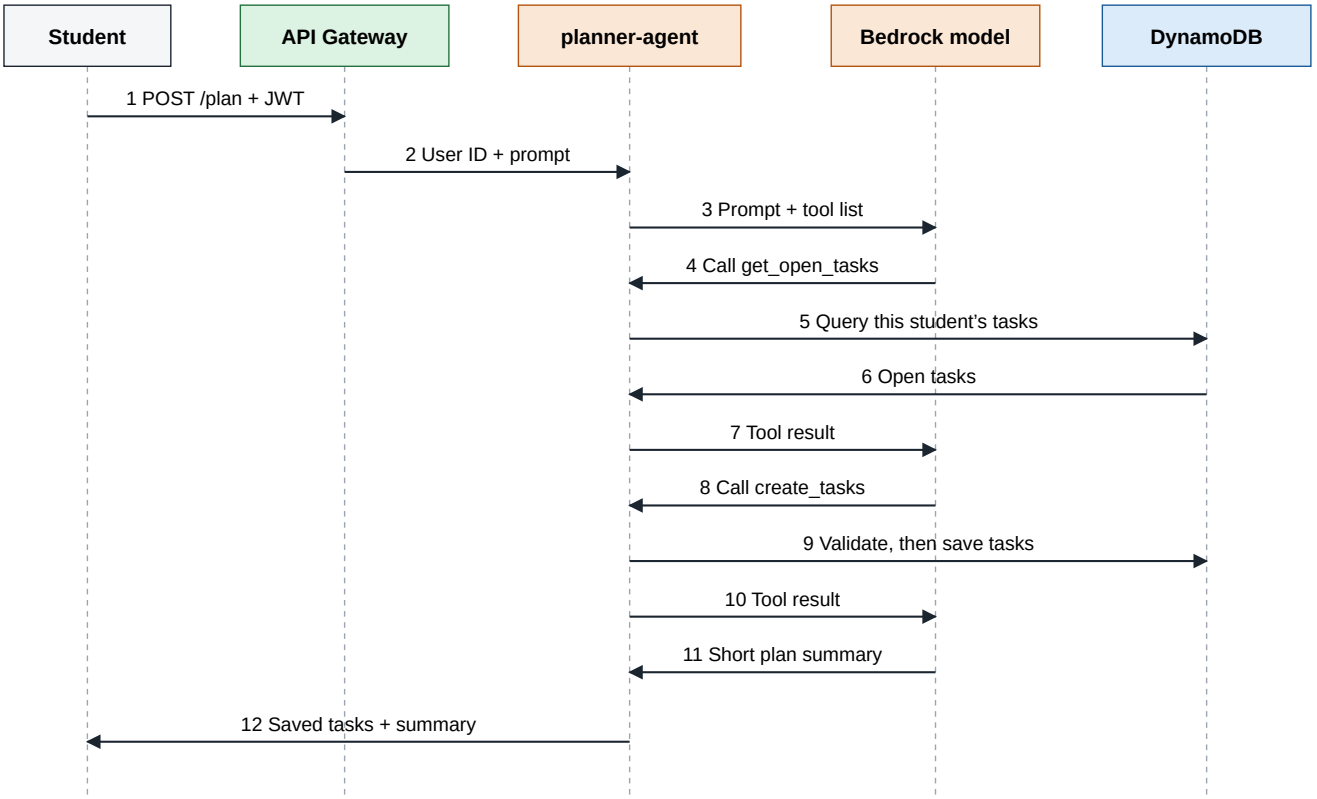


Figure 2. Example prompt: “DBMS exam on Friday, finish my lab record, and revise two hours a day.”

Agent tools

Tool	What Lambda does
`get_open_tasks`	Reads only the logged-in student’s unfinished tasks, so the plan avoids duplicates and clashes.
`create_tasks`	Checks each task’s title, date, priority and estimate, then saves it with `source = agent`.

Safety limits

- **User ID:** it always comes from the verified JWT, never from the model. A tricky prompt cannot write to another student’s account.
- **Spending:** a limit of 3 model calls per request, 1,000 output tokens per call and 20 tasks per plan.
- **Timing:** Lambda times out after 28 seconds, because an HTTP API integration cannot exceed 30 seconds.
- **Rate limit:** `/plan` has its own low throttle in API Gateway, set at 2 requests per second.

> **Model:** start with Claude Haiku 4.5 through the India inference profile `in.anthropic.claude-haiku-4-5-20251001-v1:0`, which routes between Mumbai and Hyderabad. The Converse API keeps the code the same across models, so if billing blocks Claude, Aadesh can pick an Amazon Nova profile and change one parameter.

3. API and data

Every route requires a valid Cognito JWT. CloudFormation defines each route, and the Lambda functions stay stateless so that any copy can serve any request.

Route	Purpose	Lambda
`GET /tasks`, `POST /tasks`	List tasks, add a task	`tasks-api`
`PATCH /tasks/{id}`, `DELETE /tasks/{id}`	Edit, complete or delete a task	`tasks-api`
`GET /progress`	Completion count and daily streak	`tasks-api`
`POST /tasks/{id}/attachment-url`	Get a 5-minute upload URL	`tasks-api`
`POST /plan`	Turn a prompt into saved tasks	`planner-agent`

CORS: the API and S3 bucket allow all origins, because the Amplify URL does not exist until the last stack and login uses bearer tokens rather than cookies. This keeps deployment simple for a course project.

DynamoDB table: `taskcloud-tasks`

Billing is on demand, so an idle project costs almost nothing. A student's tasks can be read with a single query on their user ID.

Attribute	Type	Notes
`userId`	Partition key	Cognito `sub` from the verified JWT
`taskId`	Sort key	Generated unique ID
`title`, `dueDate`	String	Task name and ISO date
`priority`, `status`	String	`low` / `medium` / `high` and `todo` / `done`
`estMinutes`	Number	Estimated study time
`source`	String	`manual` or `agent`
`attachmentKey`	String	S3 object key, if a file is attached
`completedAt`	String	Used to calculate progress and streaks

Consistency: `GET /tasks` uses a strongly consistent read, so a newly saved task appears immediately. `GET /progress` uses the default eventually consistent read, because a short delay does not matter there.

S3 bucket: attachments

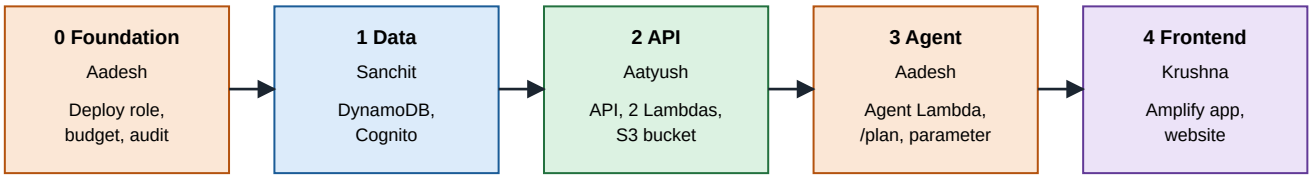
- Block Public Access is on, ACLs stay disabled and default encryption is on.
- Objects use the key `userId/taskId/filename`.
- An object-created event triggers `attachment-indexer`, matching the Lambda and S3 case study in the syllabus.

Frontend

The frontend is assumed to be a static React and Vite build in `dist/`. Amplify's manual deployment does not support server-side rendering. Krushna's script writes the API and Cognito values into `config.json`, builds the site, uploads a zip and starts the deployment.

4. Infrastructure as Code and team split

Five CloudFormation stacks must deploy in this order, because each stack exports values that the later stacks import.



Delete in the reverse order: stack 4 first, stack 0 last. Empty the S3 buckets before deleting their stacks.

Figure 3. Stack names are `taskcloud-foundation`, `-data`, `-api`, `-agent` and `-frontend`.

Who does what

Person	Order	Script	Deploys	Difficulty
Aadesh	0	`0_foundation.sh`	IAM setup, ACL, deploy role, budget, CloudTrail, model check	Hardest
Sanchit	1	`1_data.sh`	DynamoDB table, Cognito user pool	Medium
Aatyush	2	`2_api.sh`	API Gateway, `tasks-api`, `attachment-indexer`, S3 bucket and event	Medium
Aadesh	3	`3_agent.sh`	`planner-agent`, `/plan` route, model parameter, end-to-end test	Hardest
Krushna	4	`4_frontend.sh`	Amplify app and website	Easiest

Aadesh's checks before anyone runs a script

1. Turn on MFA for the root user, then work as an IAM admin rather than as root.
2. Add a valid payment method. In Bedrock, submit Anthropic's use-case details and test the model once in the playground from Mumbai.
3. Run the command below, and use it to confirm the final model profile:

```
aws bedrock list-inference-profiles --region ap-south-1 --type-equals SYSTEM_DEFINED \
  --query "inferenceProfileSummaries[?status=='ACTIVE'].inferenceProfileId"
```

> **Billing risk:** Claude on Bedrock is billed through AWS Marketplace, so promotional credits may not cover it. Indian (AISPL) accounts have also reported payment errors when subscribing. The AWS free account plan also limits some services. Test this first. If it fails, set the parameter to an Amazon Nova profile.

5. How each person deploys

Each person opens AWS CloudShell in Mumbai, then runs one script. Aadesh shares the code through a Git repository or a zip file.

```
cd ~/taskcloud
./scripts/<your_script>.sh
```

What every script does

1. Prints `aws sts get-caller-identity`, which proves who ran it.
2. Runs `aws cloudformation deploy` using the deploy role `taskcloud-cfn-role`.
3. Prints the stack status, stack events and outputs.
4. Saves everything to `logs/<script>_<timestamp>.log`.

> **If a script fails:** do not fix anything by hand in the console, because the live setup would no longer match the code. Send the log to Aadesh, wait for the fix, then run the same script again.

Evidence to submit

Person	Screenshots	Test to show working
Aadesh	IAM users and group, MFA enabled, permission policies, budget, CloudTrail trail, Bedrock playground, foundation and agent stacks showing `CREATE_COMPLETE`	`POST /plan` creates tasks that appear in DynamoDB
Sanchit	Terminal with script output, data stack `CREATE_COMPLETE` and Events tab, DynamoDB table settings, Cognito user pool	`describe-table` output shows the table as `ACTIVE`
Aatyush	Terminal with script output, API stack `CREATE_COMPLETE`, API routes and authorizer, Lambda list, S3 event notification	A call without a token returns `401`; an uploaded file triggers the Lambda
Krushna	Terminal with script output, frontend stack `CREATE_COMPLETE`, Amplify app showing a successful deployment	The live Amplify URL opens the login page

Everyone also submits their log file, using CloudShell's **Actions → Download file** option.

Krushna's script

The manual deployment works in five steps, none of which need him to make a decision:

1. Deploy the `taskcloud-frontend` stack, which creates the Amplify app and branch.
2. Read the API URL and Cognito IDs from stack outputs, then write `config.json`.
3. Run `npm ci` and `npm run build`, then zip the **contents** of `dist/`, not the folder itself.
4. Call `aws amplify create-deployment`, then upload the zip to the returned URL.
5. Call `aws amplify start-deployment`, wait for success, then print the site URL.

This route uses Amplify's upload URL. It avoids Amplify's S3-bucket deployment option, which would require turning S3 ACLs on.

6. Access Control List (ACL)

Aadesh creates this setup. In AWS, it is enforced with IAM policies rather than S3 object ACLs: new buckets disable those by default and AWS recommends keeping them off. There is also no VPC here, so no network ACL is needed.

Team access

Resource or action	Aadesh (`aadesh-admin`)	Sanchit (`sanchit`)	Aatyush (`aatyush`)	Krushna (`krushna`)
Billing, budgets, IAM users and policies	**Full**	None	None	None
Bedrock playground and model setup	**Full**	None	None	None
AWS CloudShell	**Use**	**Use**	**Use**	**Use**
View CloudFormation stacks and events	**Full**	Read	Read	Read
Pass the deploy role to CloudFormation	**Yes**	**Yes**	**Yes**	**Yes**
Deploy foundation and agent stacks	**Full**	None	None	None
Deploy data stack	**Full**	**Deploy**	None	None
Deploy API stack	**Full**	None	**Deploy**	None
Deploy frontend stack	**Full**	None	None	**Deploy**
Upload Lambda code to artifact bucket	**Full**	None	**Upload**	None
Create and start Amplify deployments	**Full**	None	None	**Deploy**
Task data in DynamoDB and files in S3	**Full**	None	None	None

How to build it: create one group, `TaskCloud-Deployers`, for the shared rows. Then add one small inline policy per person, limited to their stack's ARN. All users need console access and MFA, but no access keys.

Service access

Role	Used by	Allowed actions
`taskcloud-cfn-role`	CloudFormation	Create, update and delete only the service types listed in this document
`tasks-api-role`	`tasks-api`	Item read and write on the table; `s3:PutObject` on attachments; write logs
`indexer-role`	`attachment-indexer`	`dynamodb:UpdateItem` on the table; write logs
`planner-role`	`planner-agent`	`bedrock:InvokeModel` on the chosen profile and its models; `ssm:GetParameter` on `/taskcloud/modelId`; query and write on the table; write logs

Resource rules: API Gateway rejects any request without a valid JWT. Cognito public sign-up is off, so Aadesh creates the demo accounts. S3 blocks public access and uses 5-minute pre-signed URLs. Browsers cannot reach DynamoDB directly.

> **Known limit:** a person who may pass the deploy role could, in theory, use it through their own stack to create other allowed resources. That is acceptable for a trusted team of four, and CloudTrail records it. Delete all IAM users after the project.

7. Security, cost and syllabus map

Shared responsibility

AWS secures	The team secures
Data centres, hardware, managed service software, Lambda runtime patching	IAM users and policies, MFA, API authorization, app code, data access rules, cost limits

Traffic uses HTTPS throughout. S3 and DynamoDB encrypt stored data with their default AWS-managed encryption. CloudTrail records account activity, and CloudWatch Logs holds Lambda output.

Cost controls

- Set an AWS Budgets alert before deployment. We suggest US$5, with an email at 80%.
- Keep public sign-up off, and use the `/plan` limits from section 2.
- Delete the stacks after the final demo, so S3 storage and CloudTrail logs stop adding cost.

Syllabus coverage

Syllabus topic	Where it appears
Networks: TCP, HTTPS, firewalls	All client traffic uses HTTPS over the internet; API Gateway rejects unauthorized requests
Storage: object storage	S3 attachments and the Lambda artifact bucket
Linux and shell scripting	Bash scripts run in CloudShell
Service and deployment models	Public cloud; students get SaaS; the team builds on PaaS and serverless services
AWS accounts, IAM, budgets, CLI	Sections 4 and 6
S3, Systems Manager	Attachments; Parameter Store model setting
IaC and serverless	CloudFormation stacks; Lambda triggered by an S3 event
Security and compliance	Shared responsibility, roles, MFA, encryption, CloudTrail, audit evidence
Databases and consistency	DynamoDB with both strong and eventual reads
Cloud-native and event-driven	Stateless functions, separate services, event-driven indexer

Left out on purpose: EC2, VPC, load balancers, RDS, Redshift, SNS, SQS, containers and GuardDuty. TaskCloud does not need them, and leaving them out keeps it simple and cheap.

Sources, checked 6 November 2026

- Bedrock Agents Classic maintenance mode
- Claude Haiku 4.5 model card and Claude in India on Bedrock
- Bedrock model access, using inference profiles and profile support
- Amplify manual deployments and CreateDeployment API
- S3 Object Ownership and ACLs
- HTTP API quotas and Lambda runtimes
- re:Post on Bedrock credits and re:Post on AISPL subscriptions; community reports, not official policy